Collections in Java

The **Collection in Java** is a framework that provides an architecture to store and manipulate the group of objects.

Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.

Java Collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).

What is Collection in Java

A Collection represents a single unit of objects, i.e., a group.

What is a framework in Java

- It provides readymade architecture.
- It represents a set of classes and interfaces.
- It is optional.

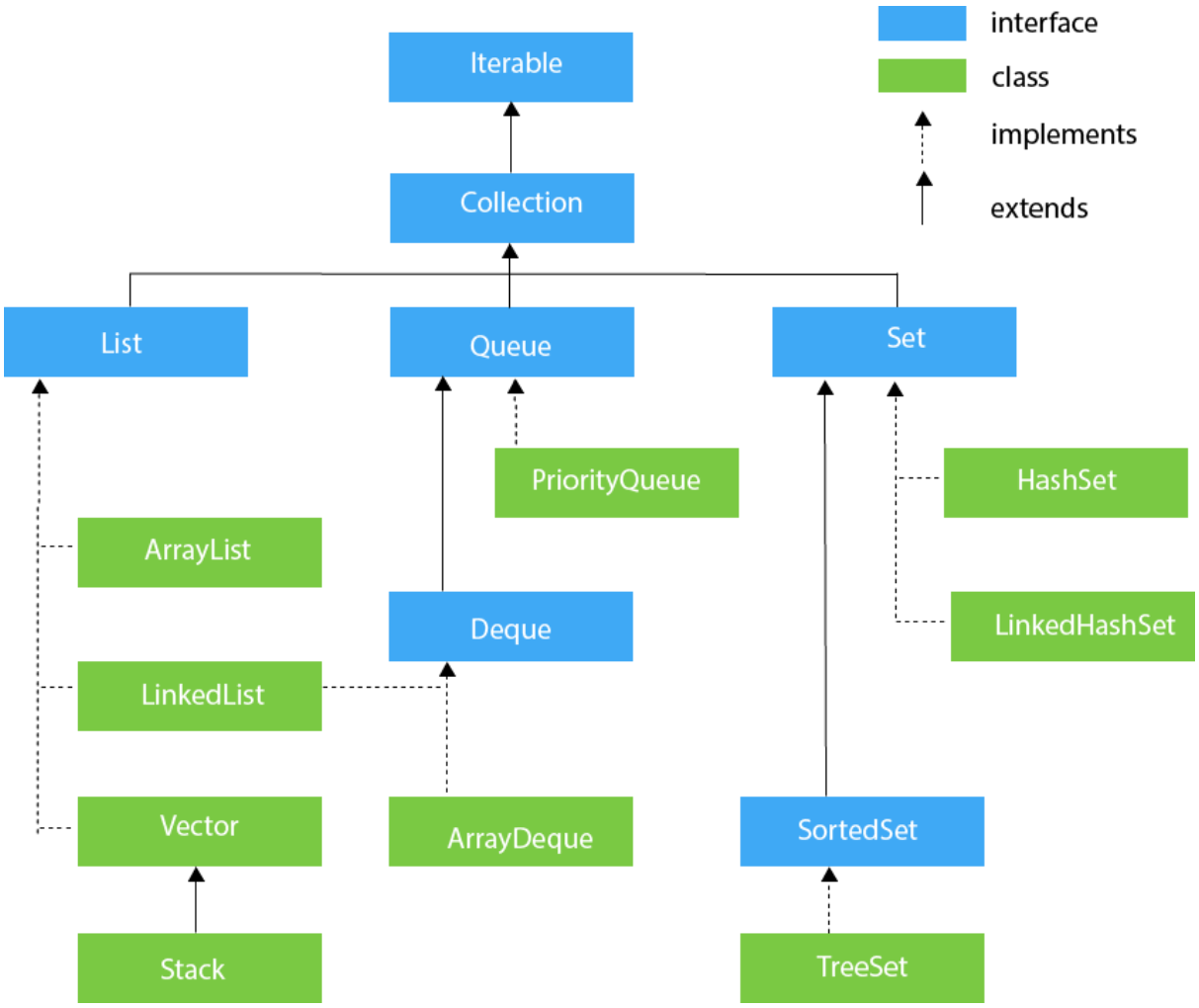
What is Collection framework

The Collection framework represents a unified architecture for storing and manipulating a group of objects. It has:

1. Interfaces and its implementations, i.e., classes
2. Algorithm

Hierarchy of Collection Framework

Let us see the hierarchy of Collection framework. The **java.util** package contains all the classes and interfaces for the Collection framework.



Methods of Collection interface

There are many methods declared in the Collection interface. They are as follows:

No.	Method	Description
1	public boolean add(E e)	It is used to insert an element in this collection.
2	public boolean addAll(Collection<? extends E> c)	It is used to insert the specified collection elements in the invoking collection.
3	public boolean remove(Object element)	It is used to delete an element from the collection.

4	public boolean removeAll(Collection<?> c)	It is used to delete all the elements of the specified collection from the invoking collection.
5	default boolean removeIf(Predicate<? super E> filter)	It is used to delete all the elements of the collection that satisfy the specified predicate.
6	public boolean retainAll(Collection<?> c)	It is used to delete all the elements of invoking collection except the specified collection.
7	public int size()	It returns the total number of elements in the collection.
8	public void clear()	It removes the total number of elements from the collection.
9	public boolean contains(Object element)	It is used to search an element.
10	public boolean containsAll(Collection<?> c)	It is used to search the specified collection in the collection.
11	public Iterator iterator()	It returns an iterator.
12	public Object[] toArray()	It converts collection into array.
13	public <T> T[] toArray(T[] a)	It converts collection into array. Here, the runtime type of the returned array is that of the specified array.
14	public boolean isEmpty()	It checks if collection is empty.
15	default Stream<E> parallelStream()	It returns a possibly parallel Stream with the collection as its source.
16	default Stream<E> stream()	It returns a sequential Stream with the collection as its source.
17	default Spliterator<E> spliterator()	It generates a Spliterator over the specified elements in the collection.

18	public boolean equals(Object element)	It matches two collections.
19	public int hashCode()	It returns the hash code number of the collection.

Iterator interface

Iterator interface provides the facility of iterating the elements in a forward direction only.

Methods of Iterator interface

There are only three methods in the Iterator interface. They are:

1.	public boolean hasNext()	It returns true if the iterator has more elements otherwise it returns false.
2	public Object next()	It returns the element and moves the cursor pointer to the next element.
3	public void remove()	It removes the last elements returned by the iterator. It is less used.

Iterable Interface

The Iterable interface is the root interface for all the collection classes. The Collection interface extends the Iterable interface and therefore all the subclasses of Collection interface also implement the Iterable interface.

It contains only one abstract method. i.e.,

1. Iterator<T> iterator()

It returns the iterator over the elements of type T.

Collection Interface

The Collection interface is the interface which is implemented by all the classes in the collection framework. It declares the methods that every collection will have. In other words, we can say that the Collection interface builds the foundation on which the collection framework depends.

Some of the methods of Collection interface are Boolean add (Object obj), Boolean addAll (Collection c), void clear(), etc. which are implemented by all the subclasses of Collection interface.

List Interface

List interface is the child interface of Collection interface. It inhibits a list type data structure in which we can store the ordered collection of objects. It can have duplicate values.

List interface is implemented by the classes ArrayList, LinkedList, Vector, and Stack.

To instantiate the List interface, we must use :

```
List <data-type> list1= new ArrayList();
List <data-type> list2 = new LinkedList();
List <data-type> list3 = new Vector();
List <data-type> list4 = new Stack();
ArrayList <data-type> list1= new ArrayList();
```

ArrayList

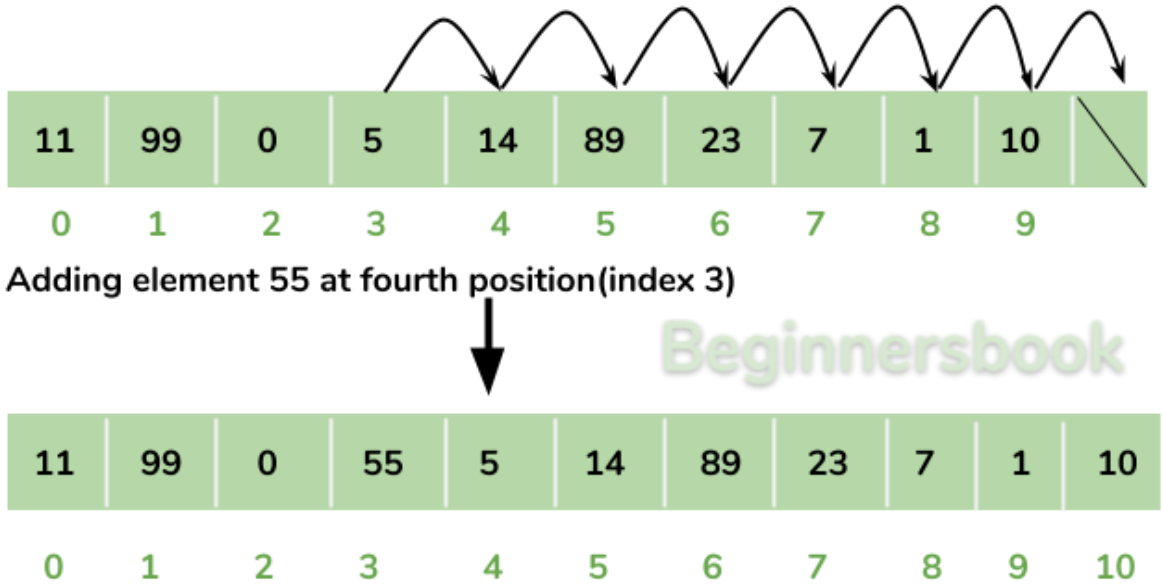
The ArrayList class implements the List interface. It uses a dynamic array to store the duplicate element of different data types. The ArrayList class maintains the insertion order and is non-synchronized. The elements stored in the ArrayList class can be randomly accessed.

Why ArrayList is better than Array?

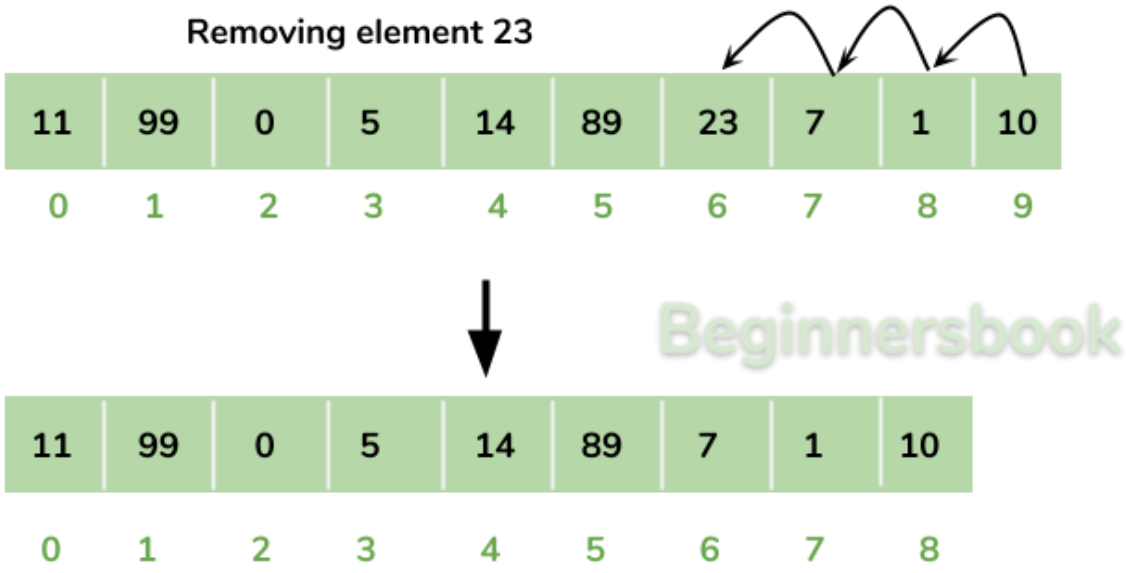
The limitation with array is that it has a **fixed length** so if it is full you cannot add any more elements to it, likewise if there are number of elements gets removed from it, the memory consumption would be the same as it doesn't shrink.

On the other **ArrayList can dynamically grow and shrink** after addition and removal of elements (See the images below). Apart from these benefits ArrayList class enables us to use predefined methods of it which makes our task easy. Let's see the diagrams to understand the addition and removal of elements from ArrayList and then we will see the programs.

Adding Element in ArrayList at specified position:



Removing Element from ArrayList:



Methods of ArrayList class

1) **add(Object o)**: This method adds an object o to the arraylist.

obj.add("hello");

This statement would add a string hello in the arraylist at last position.

2) **add(int index, Object o)**: It adds the object o to the array list at the given index.

obj.add(2, "bye");

It will add the string bye to the 2nd index (3rd position as the array list starts with index 0) of array list.

3) **remove(Object o)**: Removes the object o from the ArrayList.

obj.remove("Chaitanya");

This statement will remove the string “Chaitanya” from the ArrayList.

4) **remove(int index)**: Removes element from a given index.

obj.remove(3);

It would remove the element of index 3 (4th element of the list – List starts with o).

5) **set(int index, Object o)**: Used for updating an element. It replaces the element present at the specified index with the object o.

obj.set(2, "Tom");

It would replace the 3rd element (index =2 is 3rd element) with the value Tom.

6) **int indexOf(Object o)**: Gives the index of the object o. If the element is not found in the list then this method returns the value -1.

int pos = obj.indexOf("Tom");

This would give the index (position) of the string Tom in the list.

7) **Object get(int index)**: It returns the object of list which is present at the specified index.

String str= obj.get(2);

Function get would return the string stored at 3rd position (index 2) and would be assigned to the string “str”. We have stored the returned value in string variable because in our example we have defined the ArrayList is of String type. If you are having integer array list then the returned value should be stored in an integer variable.

8) **int size()**: It gives the size of the ArrayList – Number of elements of the list.

int numberofitems = obj.size();

9) **boolean contains(Object o)**: It checks whether the given object o is present in the array list if its there then it returns true else it returns false.

obj.contains("Steve");

It would return true if the string “Steve” is present in the list else we would get false.

10) **clear():** It is used for removing all the elements of the array list in one go. The below code will remove all the elements of ArrayList whose object is obj.

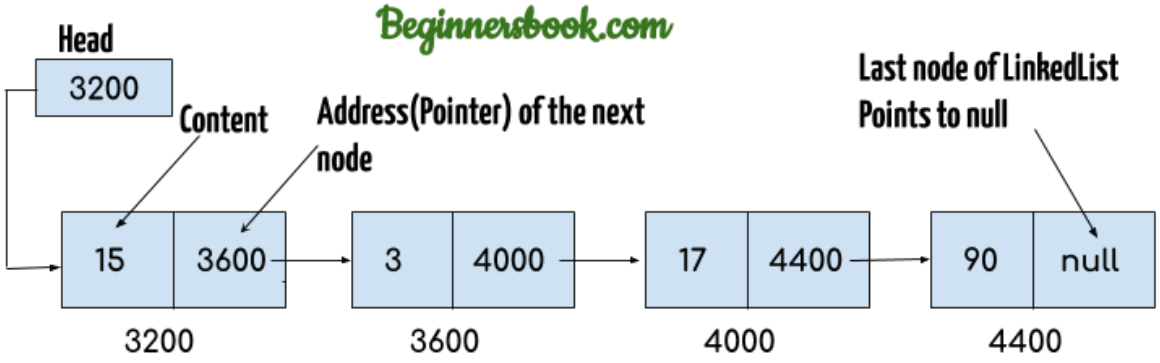
obj.clear();

LinkedList in Java

Similar to arrays in Java, **LinkedList is a linear data structure**. However LinkedList elements are not stored in contiguous locations like arrays, they are linked with each other using pointers. Each element of the LinkedList has the reference(address/pointer) to the next element of the LinkedList.

LinkedList representation

Each element in the LinkedList is called the **Node**. Each Node of the LinkedList contains two items: 1) Content of the element 2) Pointer/Address/Reference to the Next Node in the LinkedList.



Note:

1. **Head** of the LinkedList only contains the Address of the **First element** of the List.
2. The Last element of the LinkedList contains **null** in the pointer part of the node because it is the end of the List so it doesn't point to anything as shown in the above diagram.
3. The diagram which is shown above represents a **singly linked list**. There is another complex type variation of LinkedList which is called **doubly linked list**, node of a doubly linked list contains three parts: 1) Pointer to the previous node of the linked list 2) content of the element 3) pointer to the next node of the linked list.

Why do we need a Linked List?

You must be aware of the arrays which is also a linear data structure but **arrays have certain limitations such as:**

1) **Size of the array is fixed** which is decided when we create an array so it is hard to predict the number of elements in advance, if the declared size fall short then we cannot increase the size of an array and if we declare a large size array and do not need to store that many elements then it is a waste of memory.

2) Array elements **need contiguous memory locations** to store their values.

3) **Inserting an element in an array is performance wise expensive** as we have to shift several elements to make a space for the new element. For example:
Let's say we have an array that has following elements: 10, 12, 15, 20, 4, 5, 100, now

if we want to insert a new element 99 after the element that has value 12 then we have to shift all the elements after 12 to their right to make space for new element.

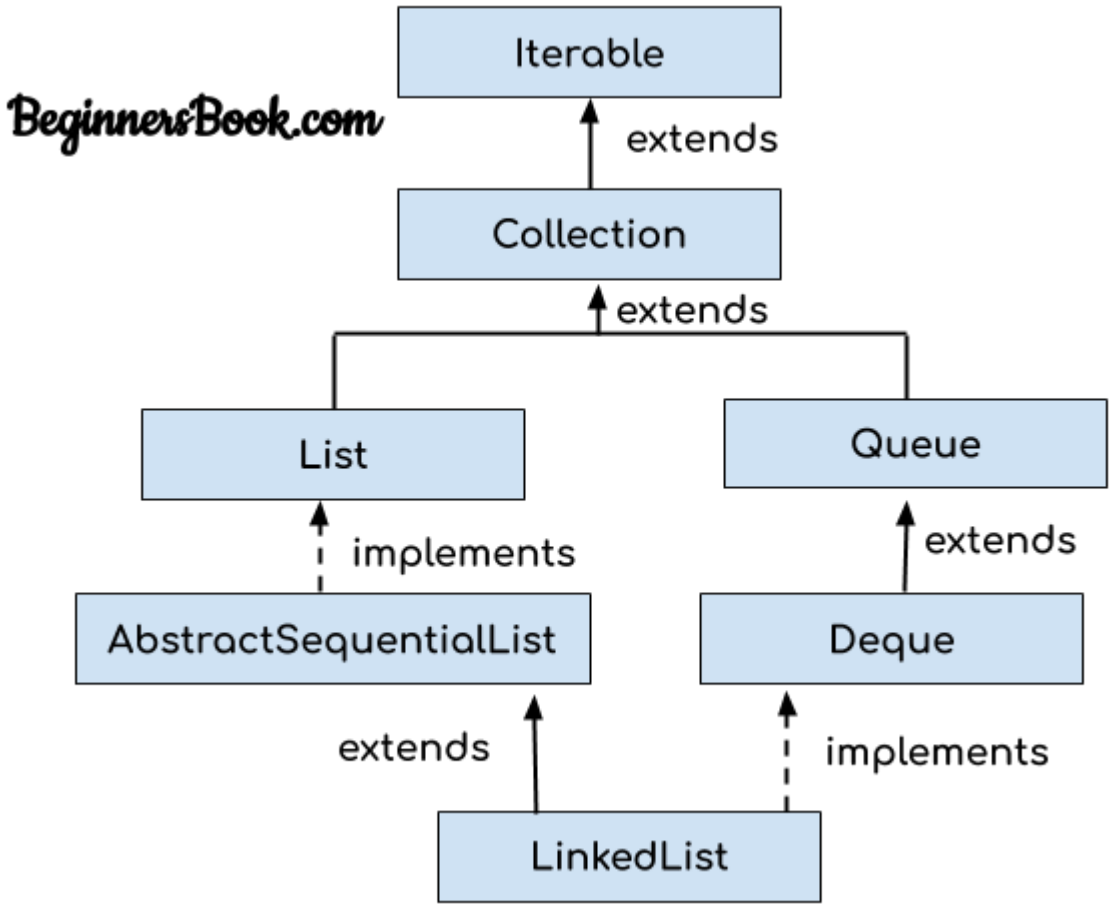
Similarly **deleting an element** from the array is also a performance wise expensive operation because all the elements after the deleted element have to be shifted left.

These limitations are handled in the Linked List by providing following features:
1. Linked list allows **dynamic memory allocation**, which means memory allocation is done at the run time by the compiler and we do not need to mention the size of the list during linked list declaration.

2. Linked list elements **don't need contiguous memory locations** because elements are linked with each other using the reference part of the node that contains the address of the next node of the list.

3. Insert and delete operations in the Linked list are not performance wise expensive because adding and deleting an element from the linked list does't require element shifting, only the pointer of the previous and the next node requires change.

Hierarchy of LinkedList class in Java



Methods of LinkedList class:

1) **boolean add(Object item)**: It adds the item at the end of the list.

```
llistobj.add("Hello");
```

It would add the string “Hello” at the end of the linked list.

2) **void add(int index, Object item)**: It adds an item at the given index of the the list.

```
llistobj.add(2, "bye");
```

This will add the string “bye” at the 3rd position(2 index is 3rd position as index starts with 0).

3) **boolean addAll(Collection c)**: It adds all the elements of the specified collection c to the list. It throws NullPointerException if the specified collection is null. Consider the below example –

```
LinkedList<String> llistobj = new LinkedList<String>();
ArrayList<String> arraylist= new ArrayList<String>();
arraylist.add("String1");
arraylist.add("String2");
llistobj.addAll(arraylist);
```
This piece of code would add all the elements of ArrayList to the LinkedList.

4) **boolean addAll(int index, Collection c)**: It adds all the elements of collection c to the list starting from a give index in the list. It throws NullPointerException if the collection c is null and IndexOutOfBoundsException when the specified index is out of the range.

```
llistobj.add(5, arraylist);
```
It would add all the elements of the ArrayList to the LinkedList starting from position 6 (index 5).

5) **void addFirst(Object item)**: It adds the item (or element) at the first position in the list.

```
llistobj.addFirst("text");
```
It would add the string “text” at the beginning of the list.

6) **void addLast(Object item)**: It inserts the specified item at the end of the list.

```
llistobj.addLast("Chaitanya");
```
This statement will add a string “Chaitanya” at the end position of the linked list.

7) **void clear()**: It removes all the elements of a list.

```
llistobj.clear();
```
8) **Object clone()**: It returns the copy of the list.

For e.g. My linkedList has four items: text1, text2, text3 and text4.

```
Object str= llistobj.clone();
 System.out.println(str);
```
Output: The output of above code would be:

[text1, text2, text3, text4]

9) **boolean contains(Object item)**: It checks whether the given item is present in the list or not. If the item is present then it returns true else false.

```
boolean var = llistobj.contains("TestString");
```
It will check whether the string “TestString” exist in the list or not.

10) **Object get(int index)**: It returns the item of the specified index from the list.

```
Object var = llistobj.get(2);
```
It will fetch the 3rd item from the list.

11) **Object getFirst()**: It fetches the first item from the list.

```
Object var = llistobj.getFirst();
```
12) **Object getLast()**: It fetches the last item from the list.

```
Object var= llistobj.getLast();
```
13) **int indexOf(Object item)**: It returns the index of the specified item.

```
llistobj.indexOf("bye");
```
14) **int lastIndexOf(Object item)**: It returns the index of last occurrence of the specified element.

```
int pos = llistobj.lastIndexOf("hello);
```
integer variable pos will be having the index of last occurrence of string “hello”.

15) **Object poll()**: It returns and removes the first item of the list.

```
Object o = llistobj.poll();
```
16) **Object pollFirst()**: same as poll() method. Removes the first item of the list.

```
Object o = llistobj.pollFirst();
```
17) **Object pollLast()**: It returns and removes the last element of the list.

```
Object o = llistobj.pollLast();
```
18) **Object remove()**: It removes the first element of the list.

```
llistobj.remove();
```
19) **Object remove(int index)**: It removes the item from the list which is present at the specified index.

```
llistobj.remove(4);
```
It will remove the 5th element from the list.

20) **Object remove(Object obj)**: It removes the specified object from the list.

```
llistobj.remove("Test Item");
```
21) **Object removeFirst()**: It removes the first item from the list.

```
llistobj.removeFirst();
```
22) **Object removeLast()**: It removes the last item of the list.

```
llistobj.removeLast();
```
23) **Object removeFirstOccurrence(Object item)**: It removes the first occurrence of the specified item.

```
llistobj.removeFirstOccurrence("text");
```
It will remove the first occurrence of the string “text” from the list.

24) **Object removeLastOccurrence(Object item)**: It removes the last occurrence of the given element.

```
llistobj.removeLastOccurrence("String1);
```
It will remove the last occurrence of string “String1”.

25) **Object set(int index, Object item)**: It updates the item of specified index with the give value.

```
llistobj.set(2, "Test");
```
It will update the 3rd element with the string “Test”.

26) **int size()**: It returns the number of elements of the list.

```
llistobj.size();
```

Vector in Java

`Vector` implements List Interface. Like `ArrayList` it also maintains insertion order but it is rarely used in non-thread environment as it is synchronized and due to which it gives poor performance in searching, adding, delete and update of its elements.

Three ways to create vector class object:
Method 1:

```
Vector vec = new Vector();
```
It creates an empty Vector with the default initial capacity of 10. It means the Vector will be re-sized when the 11th elements needs to be inserted into the Vector. Note: By default vector doubles its size. i.e. In this case the Vector size would remain 10 till 10

insertions and once we try to insert the 11th element It would become 20 (double of default capacity 10).

Method 2:
Syntax: `Vector object= new Vector(int initialCapacity)`

```
Vector vec = new Vector(3);
```
It will create a Vector of initial capacity of 3.

Method 3:
Syntax:

```
Vector object= new vector(int initialcapacity, capacityIncrement)
```
Example:

```
Vector vec= new Vector(4, 6)
```
Here we have provided two arguments. The initial capacity is 4 and capacityIncrement is 6. It means upon insertion of 5th element the size would be 10 (4+6) and on 11th insertion it would be 16(10+6).

Commonly used methods of Vector Class:

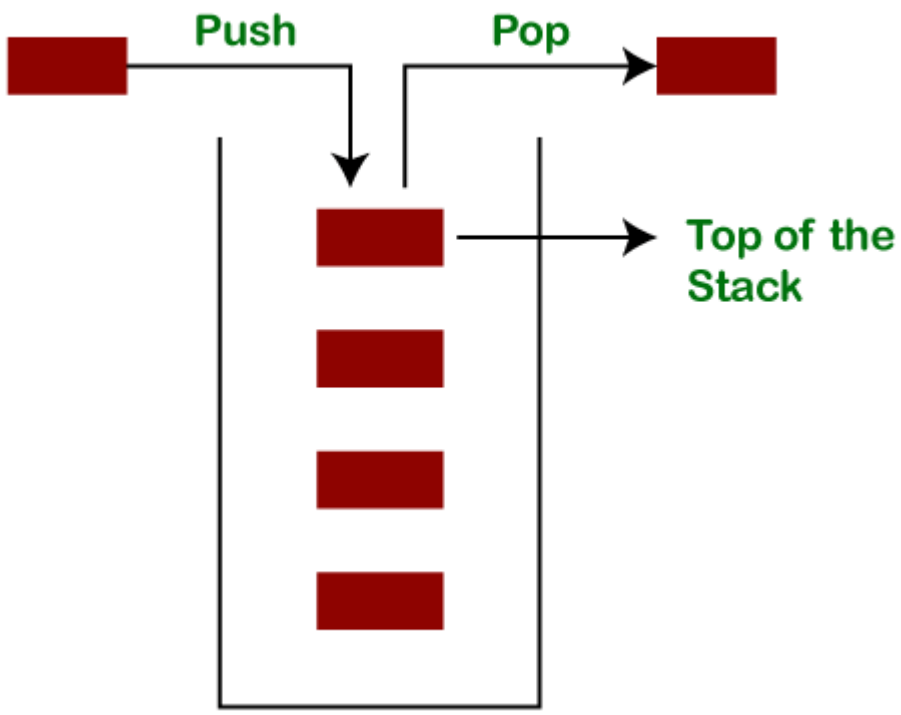
1. **void addElement(Object element):** It inserts the element at the end of the Vector.
2. **int capacity():** This method returns the current capacity of the vector.
3. **int size():** It returns the current size of the vector.
4. **void setSize(int size):** It changes the existing size with the specified size.
5. **boolean contains(Object element):** This method checks whether the specified element is present in the Vector. If the element is been found it returns true else false.
6. **boolean containsAll(Collection c):** It returns true if all the elements of collection c are present in the Vector.
7. **Object elementAt(int index):** It returns the element present at the specified location in Vector.
8. **Object firstElement():** It is used for getting the first element of the vector.
9. **Object lastElement():** Returns the last element of the array.
10. **Object get(int index):** Returns the element at the specified index.
11. **boolean isEmpty():** This method returns true if Vector doesn't have any element.
12. **boolean removeElement(Object element):** Removes the specifed element from vector.
13. **boolean removeAll(Collection c):** It Removes all those elements from vector which are present in the Collection c.

14. **void setElementAt(Object element, int index):** It updates the element of specifed index with the given element.

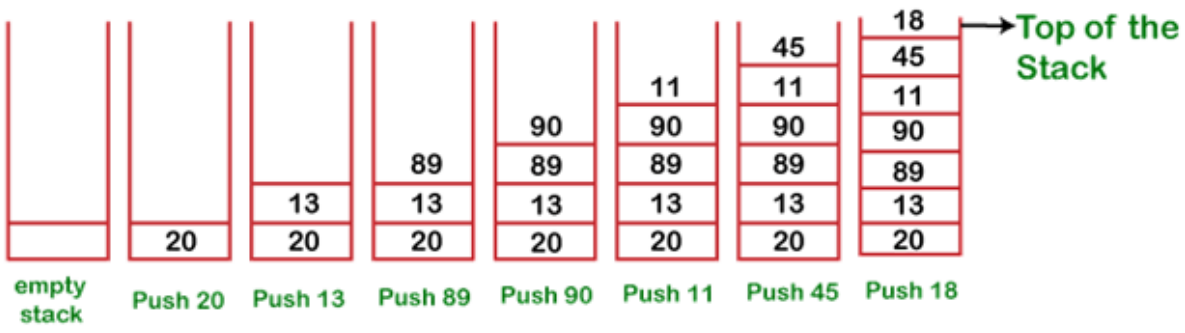
Java Stack

The **stack** is a linear data structure that is used to store the collection of objects. It is based on **Last-In-First-Out** (LIFO). Java collection framework provides many interfaces and classes to store the collection of objects. One of them is the **Stack class** that provides different operations such as push, pop, search, etc.

The stack data structure has the two most important operations that are **push** and **pop**. The push operation inserts an element into the stack and pop operation removes an element from the top of the stack. Let's see how they work on stack.

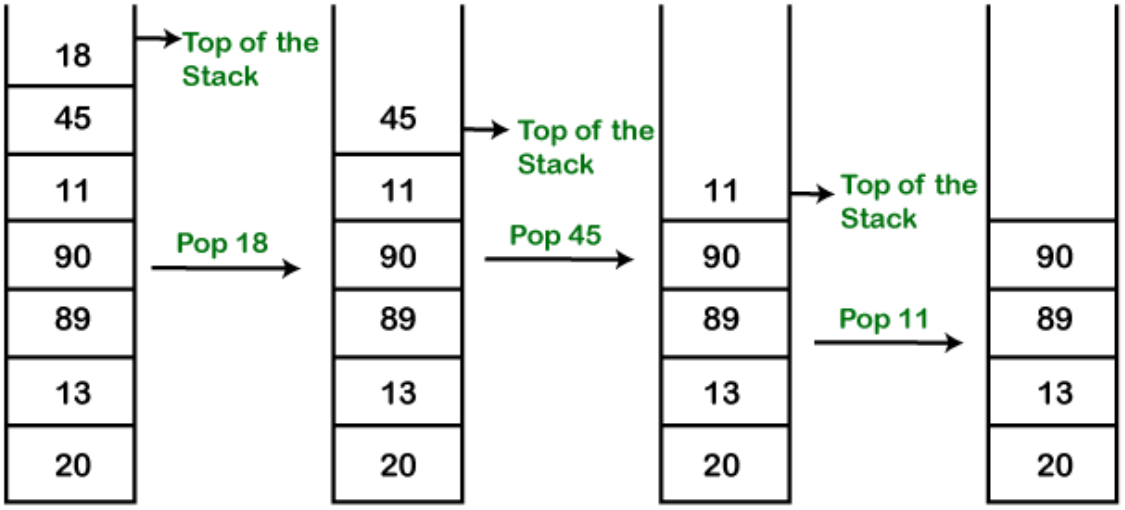


Let's push 20, 13, 89, 90, 11, 45, 18, respectively into the stack.



Push operation

Let's remove (pop) 18, 45, and 11 from the stack.



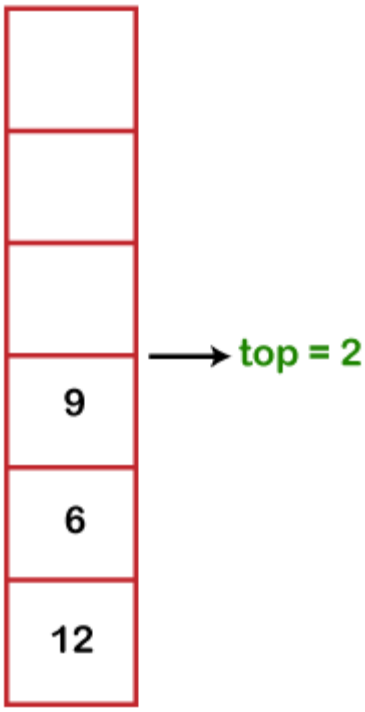
Pop operation

Empty Stack: If the stack has no element is known as an **empty stack**. When the stack is empty the value of the top variable is -1.

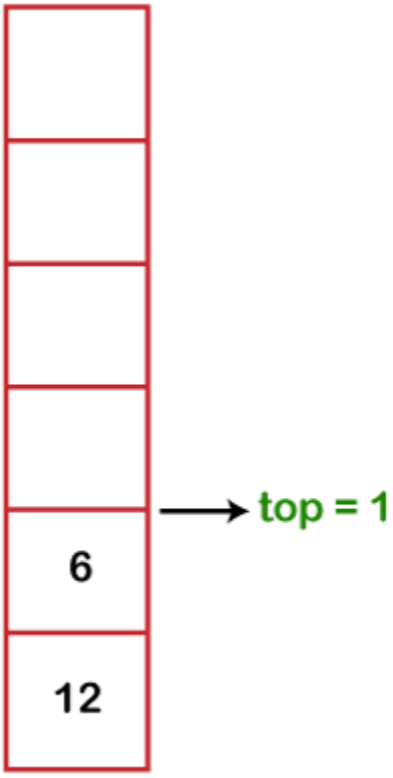


When we push an element into the stack the top is **increased by 1**. In the following figure,

- Push 12, top=0
- Push 6, top=1
- Push 9, top=2



When we pop an element from the stack the value of top is **decreased by 1**. In the following figure, we have popped 9.

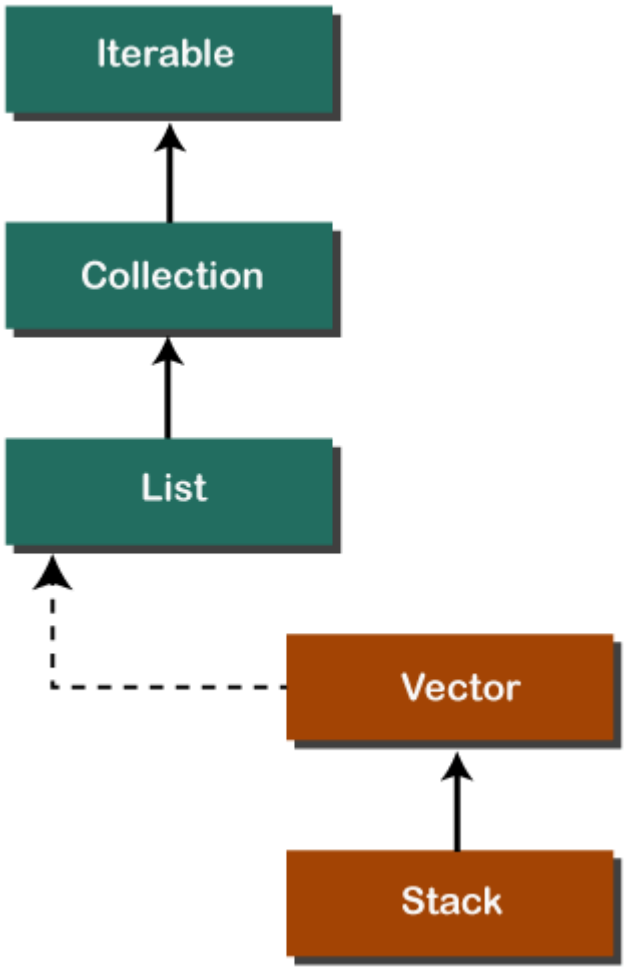


The following table shows the different values of the top.

Top value	Meaning
-1	It shows the stack is empty.
0	The stack has only an element.
N-1	The stack is full.
N	The stack is overflow.

Java Stack Class

In Java, **Stack** is a class that falls under the Collection framework that extends the **Vector** class. It also implements interfaces **List, Collection, Iterable, Cloneable, Serializable.** It represents the LIFO stack of objects. Before using the Stack class, we must import the java.util package. The stack class arranged in the Collections framework hierarchy, as shown below.



Stack Class Constructor

The Stack class contains only the **default constructor** that creates an empty stack.

1. **public** Stack()

Creating a Stack

If we want to create a stack, first, import the java.util package and create an object of the Stack class.

1. Stack stk = **new** Stack();

Or

1. Stack<type> stk = **new** Stack<>();

   Where type denotes the type of stack like Integer, String, etc.

2. Methods of the Stack Class

3. We can perform push, pop, peek and search operation on the stack. The Java Stack class provides mainly five methods to perform these operations. Along with this, it also provides all the methods of the Java Vector class.

Stack Class empty() Method

The **empty()** method of the Stack class check the stack is empty or not. If the stack is empty, it returns true, else returns false. We can also use the isEmpty() method of the Vector class.

Methods of Stack

- **public boolean empty():** is used for returns true provided Stack is empty.It returns false in case of Stack is non-empty.
- **public void push (Object):** is used for inserting the elements into the Stack.
- **public Object pop():** is used for removing Top Most elements from the Stack.
- **public Object peek():** is used for retrieving Top Most element from the Stack.
- **public int search(Object):** is used for searching an element in the Stack.If the element is found then it returns Stack relative position of that element otherwise it returns -1, -1 indicates search is unsuccessful and element is not found.

Stack Class push() Method

The method inserts an item onto the top of the stack. It works the same as the method addElement(item) method of the Vector class. It passes a parameter **item** to be pushed into the stack.

Syntax

1. **public** E push(E item)

Parameter: An item to be pushed onto the top of the stack.

Returns: The method returns the argument that we have passed as a parameter.

Stack Class pop() Method

The method removes an object at the top of the stack and returns the same object. It throws **EmptyStackException** if the stack is empty.

Syntax

1. **public** E pop()

Returns: It returns an object that is at the top of the stack.

Set in java:

A Set is a Collection that cannot contain duplicate elements. There are three main implementations of Set interface: HashSet, TreeSet, and LinkedHashSet.

HashSet, which stores its elements in a hash table, is the best-performing implementation; however it makes no guarantees concerning the order of iteration.

TreeSet, which stores its elements in a red-black tree, orders its elements based on their values; it is substantially slower than HashSet.

LinkedHashSet, which is implemented as a hash table with a linked list running through it, orders its elements based on the order in which they were inserted into the set (insertion-order).

HashSet Class

This class implements the Set interface, backed by a hash table (actually a HashMap instance). It makes no guarantees as to the iteration order of the set; in particular, it does not guarantee that the order will remain constant over time. This class permits the null element. This class is not synchronized. However it can be synchronized explicitly like this: `Set s = Collections.synchronizedSet(new HashSet(...));`

Points to Note about HashSet:

1. HashSet doesn't maintain any order, the elements would be returned in any random order.
2. HashSet doesn't allow duplicates. If you try to add a duplicate element in HashSet, the old value would be overwritten.
3. HashSet allows null values however if you insert more than one nulls it would still return only one null value.
4. HashSet is non-synchronized.
5. The iterator returned by this class is fail-fast which means iterator would throw `ConcurrentModificationException` if HashSet has been modified after creation of iterator, by any means except iterator's own remove method.

HashSet Methods:

1. **boolean add(Element e)**: It adds the element e to the list.
2. **void clear()**: It removes all the elements from the list.
3. **Object clone()**: This method returns a shallow copy of the HashSet.
4. **boolean contains(Object o)**: It checks whether the specified Object o is present in the list or not. If the object has been found it returns true else false.
5. **boolean isEmpty()**: Returns true if there is no element present in the Set.
6. **int size()**: It gives the number of elements of a Set.
7. **Boolean remove(Object o)**: It removes the specified Object o from the Set.

LinkedHashSet Class

LinkedHashSet is also an implementation of Set interface, it is similar to the HashSet and TreeSet except the below mentioned differences:

1. `HashSet` doesn't maintain any kind of order of its elements.
2. `TreeSet` sorts the elements in ascending order.
3. `LinkedHashSet` maintains the insertion order. Elements gets sorted in the same sequence in which they have been added to the Set.

TreeSet Class

`TreeSet` is similar to **HashSet** except that it sorts the elements in the ascending order while HashSet doesn't maintain any order. TreeSet allows null element but like

HashSet it doesn't allow. Like most of the other collection classes this class is also not synchronized, however it can be synchronized explicitly like this: `SortedSet s = Collections.synchronizedSortedSet(new TreeSet(...));`

Difference between HashSet and TreeSet

1) **HashSet** gives better performance (faster) than **TreeSet** for the operations like add, remove, contains, size etc. HashSet offers constant time cost while TreeSet offers log(n) time cost for such operations.

2) HashSet does not maintain any order of elements while TreeSet elements are sorted in ascending order by default.

Similarities:

1) Both HashSet and TreeSet does not hold duplicate elements, which means both of these are duplicate free.

2) If you want a sorted Set then it is better to add elements to HashSet and then **convert it into TreeSet** rather than creating a TreeSet and adding elements to it.

3) Both of these classes are non-synchronized that means they are not thread-safe and should be synchronized explicitly when there is a need of thread-safe operations.

Java Collections – Map

A Map is an object that maps keys to values. A map cannot contain duplicate keys. There are three main implementations of Map interfaces: HashMap, TreeMap, and LinkedHashMap.
HashMap: it makes no guarantees concerning the order of iteration
TreeMap: It stores its elements in a red-black tree, orders its elements based on their values; it is substantially slower than HashMap.
LinkedHashMap: It orders its elements based on the order in which they were inserted into the set (insertion-order).

HashMap in Java

HashMap is a Map based collection class that is used for storing Key & value pairs, it is denoted as HashMap<Key, Value> or HashMap<K, V>. This class makes no guarantees as to the order of the map. It is similar to the Hashtable class except that it is unsynchronized and permits nulls(null values and null key).

It is not an ordered collection which means it does not return the keys and values in the same order in which they have been inserted into the HashMap. It does not sort the stored keys and Values. You must need to import `java.util.HashMap` or its super class in order to use the HashMap class and methods.

```
import java.util.HashMap;
import java.util.Map;
import java.util.Iterator;
import java.util.Set;
public class Details {

   public static void main(String args[]) {

      /* This is how to declare HashMap */
      HashMap<Integer, String> hmap = new HashMap<Integer,
String>();

      /*Adding elements to HashMap*/
      hmap.put(12, "Chaitanya");
      hmap.put(2, "Rahul");
      hmap.put(7, "Singh");
      hmap.put(49, "Ajeet");
      hmap.put(3, "Anuj");

      /* Display content using Iterator*/
      Set set = hmap.entrySet();
      Iterator iterator = set.iterator();
      while(iterator.hasNext()) {
         Map.Entry mentry = (Map.Entry)iterator.next();
         System.out.print("key is: "+ mentry.getKey() + " & Value
is: ");
         System.out.println(mentry.getValue());
      }

      /* Get values based on key*/
      String var= hmap.get(2);
      System.out.println("Value at index 2 is: "+var);

      /* Remove values based on key*/
      hmap.remove(3);
      System.out.println("Map key and values after removal:");
      Set set2 = hmap.entrySet();
      Iterator iterator2 = set2.iterator();
      while(iterator2.hasNext()) {
          Map.Entry mentry2 = (Map.Entry)iterator2.next();
          System.out.print("Key is: "+mentry2.getKey() + " &
Value is: ");
          System.out.println(mentry2.getValue());
       }

   }
```

TreeMap in Java

TreeMap is Red-Black tree based NavigableMap implementation. It is sorted according to the natural ordering of its keys.
TreeMap class implements Map interface similar to `HashMap` class. The main difference between them is that **HashMap** is an unordered collection while TreeMap is sorted in the ascending order of its keys. TreeMap is unsynchronized collection class which means it is not suitable for thread-safe operations until unless synchronized explicitly.

LinkedHashMap

LinkedHashMap is a Hash table and linked list implementation of the Map interface, with predictable iteration order. This implementation differs from HashMap in that it maintains a doubly-linked list running through all of its entries. This linked list defines the iteration ordering, which is normally the order in which keys were inserted into the map (insertion-order). In last few tutorials we have discussed about **HashMap** and **TreeMap**. This class is different from both of them:

- `HashMap` doesn't maintain any order.
- `TreeMap` sort the entries in ascending order of keys.
- `LinkedHashMap` maintains the insertion order.

Comparable Interface

Java Comparable interface is used to order the objects of the user-defined class. This interface is found in java.lang package and contains only one method named compareTo(Object). It provides a single sorting sequence only, i.e., you can sort the elements on the basis of single data member only. For example, it may be rollno, name, age or anything else.

compareTo(Object obj) method

public int compareTo(Object obj): It is used to compare the current object with the specified object. It returns

- positive integer, if the current object is greater than the specified object.
- negative integer, if the current object is less than the specified object.
- zero, if the current object is equal to the specified object.

We can sort the elements of:

1. String objects
2. Wrapper class objects
3. User-defined class objects

Note:String class and wrapper classes implements the Comparable interface by default.So If you store the objects of string or wrapper classes in a list,set or map,iy will be Comparable by default.

```
package collections_framework;
import java.util.*;

class students implements Comparable<students>
{
	int rollno;
	String name;
	int marks;
	students(int rollno,String name,int marks)
	{
		this.rollno=rollno;
		this.name=name;
		this.marks=marks;
	}
	@Override
	public String toString() {
		return "students [rollno=" + rollno + ", name=" + name + ", marks="
				+ marks + "]";
	}
	@Override
	public int compareTo(students s)
	{
		return ((name.length()>s.name.length())?1:-1);

	}

}


public class comparableExample {

	public static void main(String[] args) {
		ArrayList<students> list=new ArrayList<>();
		list.add(new students(101, "priya", 95));
		list.add(new students(102, "trisha", 85));
		list.add(new students(103, "raj", 34));
```

```
                Collections.sort(list);

                for(students l:list)
                System.out.println(l);

        }

}
```

				6
2				
			3	
	5			
				7
2034-2038	2038-2042	2042-2046	2046	

2 3 5 6 7